Schedules & No-Tuning

Warmup, Cosine, Schedule-Free & Prodigy

Lesson 9 of the Optimizer Course

The hidden dimension: how the learning rate should change over the course of training

We've treated the learning rate as one fixed number. It almost never is. The best training runs change *the learning rate over time — gentle at first, bold in the middle, gentle again at the end. This lesson covers that schedule, why each phase matters, and the newest methods that try to remove learning-rate tuning altogether.*

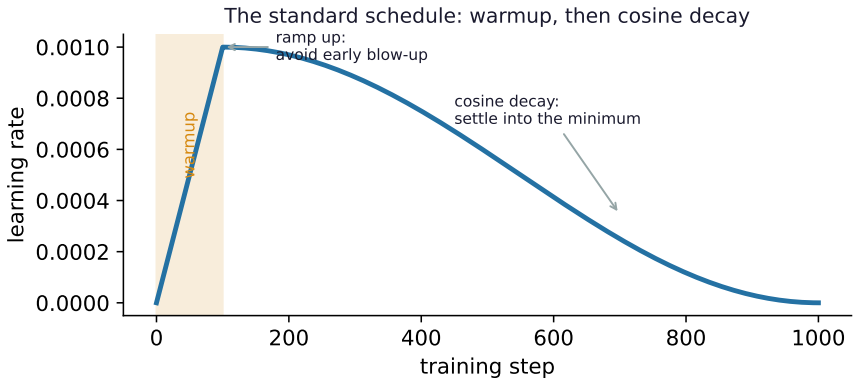
Marevlo Research

1 Why a single learning rate is wrong

Think about what training looks like at different stages:

- **At the start**, the weights are random garbage. Gradients are huge and chaotic, and the adaptive estimates (Adam's v) are based on almost no data. A big step here can fling you somewhere terrible. ⇒ you want a *small* learning rate.
- **In the middle**, things have stabilised. You're on a sensible slope with reliable gradients. ⇒ you want a *big* learning rate, to make fast progress.
- **Near the end**, you're close to the bottom of a valley. A big step now just bounces you off the walls (remember the zig-zag). ⇒ you want a *small* learning rate again, to settle gently into the minimum.

So the ideal learning rate **rises, then falls**. That shape — a brief ramp up, then a long decay — is the standard schedule for nearly every modern model.



2 Warmup: ease into it

Warmup is the opening ramp: start the learning rate near zero and increase it linearly over the first few hundred or thousand steps. Here's the concrete reason it matters, tying back to Lesson 5's bias correction:

Let's actually do the numbers

Suppose step 1 produces a big gradient spike, $g = 50$ (common with random init). Adam's bias-corrected estimates are $\hat{m} = 50$, $\hat{v} = 2500$, so $\sqrt{\hat{v}} = 50$.

- **No warmup** ($\eta = 0.001$): step $= 0.001 \cdot \frac{50}{50} = 0.001$ — and with only one noisy gradient behind that estimate, taking a full step is risky.
- **With warmup** ($\eta = 10^{-5}$ early): step $= 10^{-5} \cdot \frac{50}{50} = 0.00001$ — 100× gentler. You wait until the estimates are trustworthy before stepping boldly.

Warmup is essential for training transformers — without it, large models frequently diverge in the first few hundred steps. (Recall **RAdam** from Lesson 5: it computes how trustworthy the adaptive estimate is and effectively does this warmup *automatically*.)

3 Cosine, step, and linear decay

After warmup comes the long decay. Three shapes dominate:

Cosine decay is the most popular. It glides the learning rate from its peak down to nearly zero following a half-cosine curve — smooth all the way, no sudden jumps:

$$\eta_t = \tfrac{1}{2}\,\eta_{\max}\Big(1 + \cos\big(\pi\, t/T\big)\Big).$$

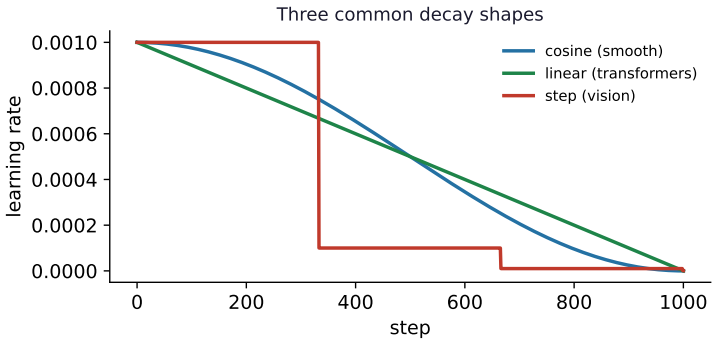
Let's actually do the numbers

With $\eta_{\max} = 0.001$ over $T = 1000$ steps:

step t	$\cos(\pi t/T)$	η_t
0	+1.000	0.001000 (full)
250	+0.707	0.000854
500	0.000	0.000500 (half)
750	−0.707	0.000146
1000	−1.000	0.000000 (zero)

A smooth descent from full rate to zero — fast early, delicate late.

Step decay drops the rate by a factor (often 10×) at fixed milestones — the classic choice for vision (it's how the original ResNets were trained). **Linear decay** draws a straight line from peak to zero — common for BERT-style transformer training.



Watch out (common confusion)

Cosine and linear decay need to know the *total* number of steps T in advance — the curve is shaped to hit zero exactly at the end. If you stop early, you were never running the schedule you thought. If you train longer than planned, the rate already hit zero and learning stopped. This "you must commit to T up front" annoyance is exactly what the next methods remove.

4 The no-tuning frontier

The learning rate (and its schedule) is the hyperparameter people waste the most time tuning. Two recent families try to delete that work entirely.

Schedule-Free (Defazio et al., 2024). Instead of a decay curve, it keeps a clever running *average* of the weights it has visited, which mathematically delivers the benefit of decay *without*

a schedule — and crucially without needing to know T in advance. You just train for as long as you like and stop whenever; the averaging has been "decaying" for you all along. It drew a lot of attention in 2024 for matching or beating tuned cosine schedules.

Prodigy / D-Adaptation. These are "learning-rate-free": they *estimate the right learning rate automatically* as training proceeds. The trick is to track how far the weights have travelled from their starting point — that distance reveals the scale of the problem, which is exactly what sets the ideal learning rate. D-Adaptation introduced the idea; Prodigy is the sharper, faster successor. In practice you set the learning rate to 1.0 and let the method figure out the real one.

When you're actually training

For most work, the safe recipe is boring and excellent: **linear warmup** over the first 2–10% of training, then **cosine decay** to near zero. That single schedule, with AdamW, trains the vast majority of models well. Reach for Schedule-Free or Prodigy when you genuinely don't know how long you'll train, or when you want to skip learning-rate tuning entirely — they're promising, increasingly trusted, but not yet the universal default.

5 What you can do now, and what's next

You can now explain why the ideal learning rate rises then falls, why warmup prevents early divergence (and how it relates to Adam's unreliable early estimates), compute a cosine schedule, and describe what Schedule-Free and Prodigy do differently.

Next — Lesson 10: Frontier & Choosing. The finale. We meet the two newest optimizers making waves — **Lion** (a startlingly simple sign-based method) and **Muon** (an orthogonalising method setting LLM training-speed records) — and then assemble everything into the single most practical thing in this course: a clear framework for *which optimizer to actually pick*, and how to diagnose a training run that's misbehaving.